

Ato Store Specification

Document ID: STORE_SPEC_V2

Status: v0.1 (Accepted)

最終更新: 2026-02-07

関連仕様: [LICENSE_SPEC.md](#), [IDENTITY_SPEC.md](#), [CAPSULE_CLI_SPEC.md](#), [DESKTOP_SPEC.md](#)

1. 目的

- **グローバル配布:** Capsule の検索・配布・販売を行うパッケージレジストリを提供する
- **Edge-first:** Cloudflare エコシステムを活用し、読み取り性能の最大化とランニングコストの最小化を実現する
- **疎結合設計:** 検索エンジン・認証基盤の将来的な差し替えを可能にする抽象化レイヤーを設ける
- **既存互換:** ato-coordinator (Go) の Registry API を完全置換し、CLI/Desktop の既存クライアントを段階的に移行する

2. スコープ

2.1 スコープ内

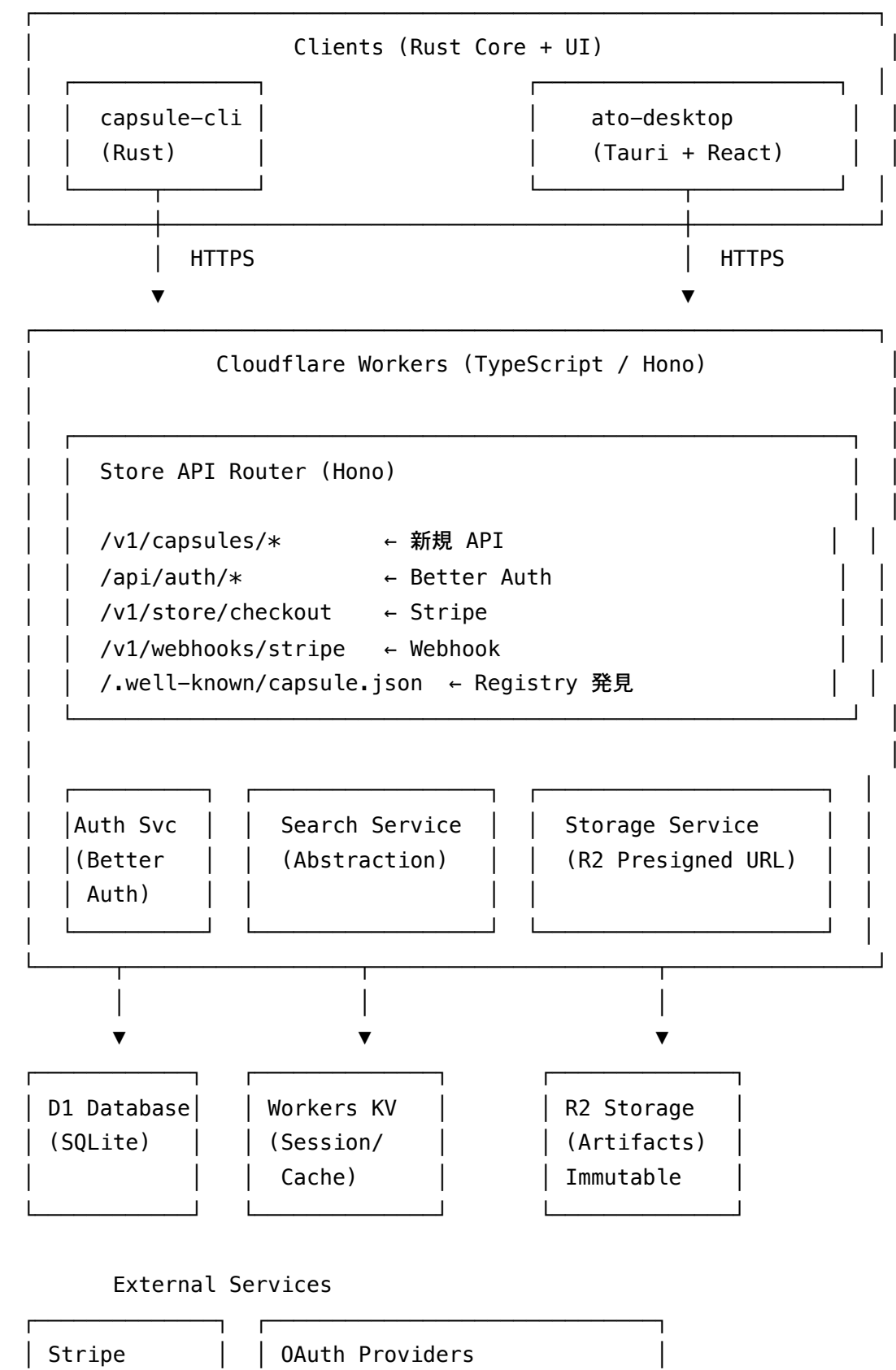
- Store API (ato-store) のエンドポイント設計
- データベーススキーマ (D1 / Drizzle ORM)
- 認証・認可 (Better Auth + OAuth)
- Publisher Identity Model (DID + OAuth Binding)
- パッケージ公開 (Immutable Release)
- 検索 (Search Abstraction Layer)
- ダウンロードと権限チェック
- R2 ストレージとPresigned URL
- Stripe 決済統合
- CLI / Desktop クライアント統合

2.2 スコープ外

- 決済の会計処理・税務 → 外部サービス (Stripe Tax)
- 開発者ダッシュボード Web UI → 別仕様
- P2P 配布 → [MAGNET_DISCOVERY.md](#)
- ライセンス検証ロジック → [LICENSE_SPEC.md](#)
- 署名と DID の詳細 → [IDENTITY_SPEC.md](#)

3. アーキテクチャ

3.1 構成図



(Checkout/ Connect)	GitHub (Publisher) Google / Apple (Buyer)
------------------------	--

3.2 技術スタック

Component	Technology	選定理由
Runtime	Cloudflare Workers	0ms コールドスタート、グローバル分散。Hono によりランタイム移植性を担保
Language	TypeScript	Web 標準準拠
Framework	Hono	超軽量・高速な Web 標準フレームワーク
Database	Cloudflare D1 (SQLite)	エッジ SQL。コスト効率と読み取り性能
ORM	Drizzle ORM	D1 ネイティブ対応、型安全なスキーマ定義
Storage	Cloudflare R2	Egress (下り転送量) 無料。S3 互換 API
Auth	Better Auth	D1/Hono/Workers 公式対応。SaaS コスト回避
Payment	Stripe	Checkout (販売) & Connect (開発者分配)
Session	Workers KV	セッション/キャッシュストア
Test	Vitest + @cloudflare/vitest-pool-workers	ローカル D1/R2 エミュレーション

3.3 コスト戦略

固定費ゼロを目指す。

項目	戦略
コンピュート	Workers Free (10万req/日) → 従量課金 (\$0.50/百万req)
データベース	D1 Free (5GB) → Paid (10GB, \$0.75/百万read)
ストレージ	R2 Free (10GB) → 従量課金 (\$0.015/GB/月)。Egress 無料
検索	D1 LIKE 検索 (追加コストなし)
決済	Stripe 手数料は売上連動 (3.6% + ¥40)

項目	戦略
固定費	\$0/月 (初期フェーズ)

4. リスクヘッジ設計 (Mitigation Strategies)

4.1 検索エンジンの抽象化 (Search Abstraction Layer)

D1 (SQLite) は FTS5 (Full-Text Search) をサポートしていない。また LIKE パターンは 50 バイト に制限されている。これに備え、検索ロジックをインターフェース化する。

```
// src/services/search/types.ts

export interface SearchOptions {
  category?: string;
  limit?: number; // default: 20, max: 50
  cursor?: string; // カーソルベースページネーション
}

export interface SearchService {
  /**
   * パッケージを検索する。
   * クエリをスペースで分割し、slug/description に対して AND 条件で検索。
   * 結果は downloads DESC でソート。
   */
  search(query: string, options: SearchOptions): Promise<CapsuleSummary[]>;

  /**
   * 新規パッケージを検索インデックスに追加する。
   */
  index(capsule: CapsuleRecord): Promise<void>;
}
```

実装ロードマップ:

Phase	実装	条件
初期	SimpleSqlSearch (D1 LIKE)	パッケージ数 < 数千件
Phase 2	MeilisearchService または CF Vectorize	検索品質が問題になった時点

SimpleSqlSearch のロジック:

- 1. クエリをスペースで分割 ("ai chat" → ["ai", "chat"])
- 2. 各単語を 50 バイト以内にトランケート (D1 LIKE 制限)
- 3. slug LIKE '%word%' OR description LIKE '%word%' を AND 結合
- 4. ORDER BY downloads DESC でソート
- 5. LIMIT 50 + カーソルベースページネーション

4.2 認証の安全弁 (Auth Safety Valve)

Better Auth の運用リスクと OAuth プロバイダのセキュリティを考慮し、以下の制約を設ける。

パスワード認証 (Email/Pass) は無効化する。

User Role	Allowed Providers	Policy
Publisher	GitHub ONLY	開発者の身元確認、Organization 情報の取得のため必須
Buyer	Google / Apple	購入専用アカウント。GitHub を持たない一般層向け

- Publisher 権限を持つには、必ず GitHub 連携が必要
- Buyer アカウントでも、後から GitHub をリンクすることで Publisher に昇格可能

Better Auth の検証項目:

- ☐ Cloudflare Workers 上での動作確認 (D1 直接接続)
- ☐ Session 管理の KV 利用時のコールドスタート影響
- ☐ トークンサイズ (Cookie 制限: 4KB)
 - ダメなら: hono/oauth-providers への即時ピボット

4.3 R2 Presigned URL セキュリティ

項目	値	理由
Download TTL	300 秒 (5分)	ダウンロード直前に発行するため十分
Upload TTL	300 秒 (5分)	CLI が即座にアップロードするため
Content-Type 制限	application/octet-stream	不正ファイルタイプの防止

5. Publisher Identity Model (DID + OAuth)

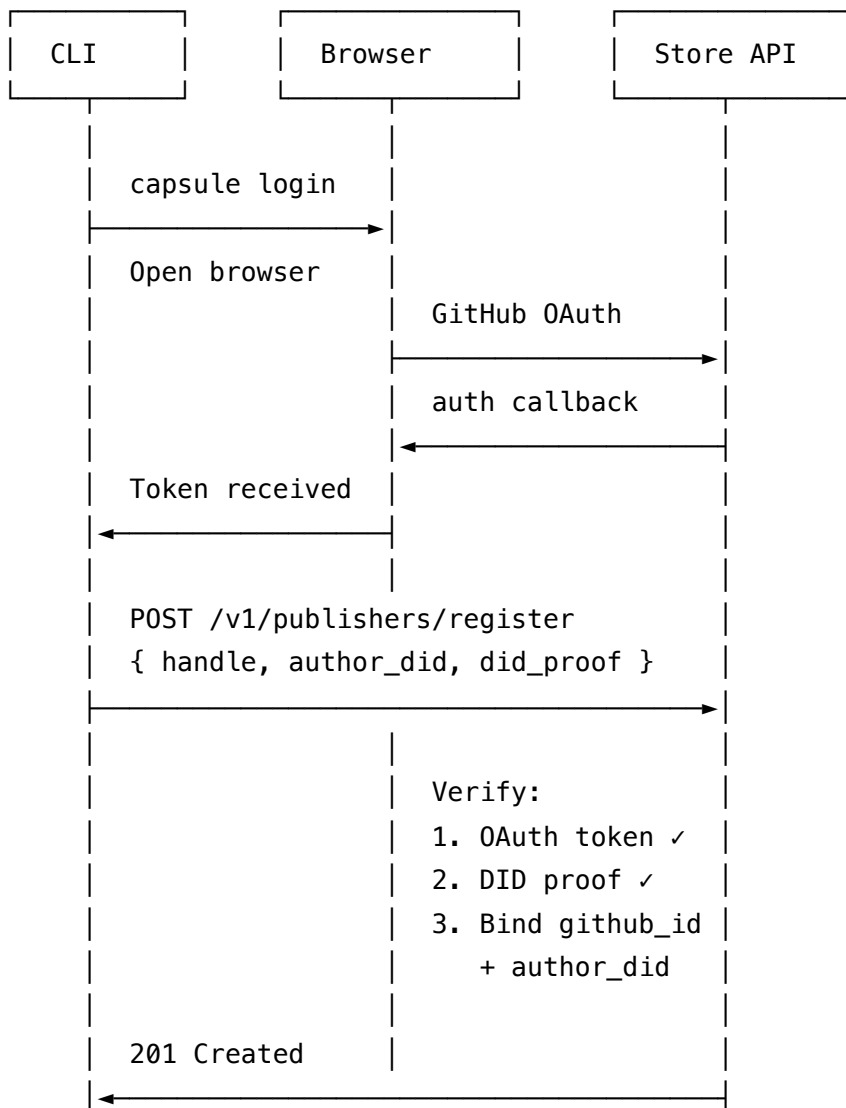
5.1 設計原則

「社会的信用 (GitHub OAuth)」 と 「暗号的署名 (DID)」 の **Binding** (紐付け) を publishers テーブルで行う。

これは Ato のセキュリティモデルの核であり、以下の2つを両立する:

1. **GitHub OAuth**: 開発者の社会的アイデンティティ (Organization, リポジトリ実績)
2. **DID (did:key:z6Mk...)**: パッケージ署名の暗号的証明 (Ed25519)

5.2 Publisher 登録フロー



did_proof の構造:

```
{
  "did": "did:key:z6MkhaXgBZDvotDkL5257faiztiGiC2QtKLGpbnnEGta2doK",
  "timestamp": "2026-02-07T00:00:00Z",
  "signature": "<base64-ed25519-signature-of-timestamp>"
}
```

サーバーは did から Ed25519 公開鍵を抽出し、timestamp に対する signature を検証する。タイムスタンプは ±5 分以内であること。

5.3 Publish 時の 2 段階認証

Publish リクエストは、以下の **両方** がパスした場合のみ受理する:

1. **OAuth Token 検証**: Authorization: Bearer <oauth_token> ヘッダー
2. **DID 署名検証**: リクエストボディの signature フィールドを publishers.author_did の公開鍵で Ed25519 検証

Authorization: Bearer <oauth_token> ← 社会的信用
Content-Type: application/json

```
{
  "slug": "my-capsule",
  "version": "1.0.0",
  "manifest": { ... },
  "signature": {                                ← 暗号的証明
    "algorithm": "Ed25519",
    "public_key": "did:key:z6Mk...",
    "content_hash": "blake3:a1b2c3...",
    "signature": "<base64>",
    "signed_at": 1738886400
  }
}
```

6. データベーススキーマ (Drizzle / D1)

6.1 Auth テーブル (Better Auth 標準)

Better Auth が自動生成する 4 テーブル:

- user: id, email, name, image, created_at, updated_at
- session: id, user_id, token, expires_at, ip_address, user_agent
- account: id, user_id, provider_id (github/google/apple), account_id
- verification: id, identifier, value, expires_at

6.2 Registry テーブル

-- publishers: パッケージ公開者

```
CREATE TABLE publishers (  
  id          TEXT PRIMARY KEY,          -- ULID  
  handle      TEXT UNIQUE NOT NULL,      -- "ato-core" (kebab-case, max 63)  
  user_id     TEXT NOT NULL REFERENCES user(id),  
  
  -- Identity Binding  
  github_id   INTEGER UNIQUE NOT NULL,   -- OAuth Identity  
  author_did  TEXT UNIQUE NOT NULL,      -- Crypto Identity (did:key:z6Mk...)  
  
  verified    INTEGER NOT NULL DEFAULT 0, -- DID 所有確認済み (boolean)  
  created_at  TEXT NOT NULL DEFAULT (datetime('now')),  
  updated_at  TEXT NOT NULL DEFAULT (datetime('now'))  
);
```

-- capsules: パッケージメタデータ

```
CREATE TABLE capsules (  
  id          TEXT PRIMARY KEY,          -- ULID  
  publisher_id TEXT NOT NULL REFERENCES publishers(id),  
  slug        TEXT UNIQUE NOT NULL,      -- "my-capsule" (kebab-case, max 63)  
  
  -- Metadata (capsule.toml 由来)  
  name        TEXT NOT NULL,            -- 表示名  
  description  TEXT NOT NULL DEFAULT '', -- 説明文 (max 500)  
  category    TEXT NOT NULL DEFAULT 'other',  
  type        TEXT NOT NULL DEFAULT 'app', -- app | cli | library  
  homepage    TEXT,                    -- URL  
  repository  TEXT,                    -- URL  
  license_spdx TEXT,                    -- "MIT", "Apache-2.0" 等  
  
  -- Commerce  
  price        INTEGER NOT NULL DEFAULT 0, -- 価格 (cents)。0 = 無料  
  currency     TEXT NOT NULL DEFAULT 'usd', -- ISO 4217  
  stripe_product_id TEXT,                -- Stripe Product ID  
  
  -- Stats  
  downloads    INTEGER NOT NULL DEFAULT 0,  
  
  created_at  TEXT NOT NULL DEFAULT (datetime('now')),  
  updated_at  TEXT NOT NULL DEFAULT (datetime('now'))  
);
```

```

-- releases: バージョン付きリリース (Immutable)
CREATE TABLE releases (
  id          TEXT PRIMARY KEY,          -- ULID
  capsule_id  TEXT NOT NULL REFERENCES capsules(id),
  version     TEXT NOT NULL,            -- SemVer "1.0.0"

  -- Artifact
  artifact_key TEXT NOT NULL,           -- R2 オブジェクトキー ("capsules/<id>/<version>"
  content_hash TEXT NOT NULL,           -- "blake3:a1b2c3..."
  size_bytes   INTEGER NOT NULL,

  -- Manifest snapshot (capsule.toml の JSON 変換)
  manifest     TEXT NOT NULL,           -- JSON

  -- Capabilities (manifest から抽出)
  capabilities TEXT NOT NULL DEFAULT '[]', -- JSON 配列

  -- Signature
  signature     TEXT NOT NULL,          -- JSON (Ed25519 署名)
  signature_status TEXT NOT NULL DEFAULT 'verified', -- verified | unverified

  -- Status
  status        TEXT NOT NULL DEFAULT 'published', -- published | yanked

  -- Immutability: 同一 capsule_id + version の INSERT は拒否
  UNIQUE(capsule_id, version),

  created_at    TEXT NOT NULL DEFAULT (datetime('now'))
);

-- licenses: 購入ライセンス
CREATE TABLE licenses (
  id          TEXT PRIMARY KEY,          -- ULID
  user_id     TEXT NOT NULL REFERENCES user(id),
  capsule_id  TEXT NOT NULL REFERENCES capsules(id),

  -- Stripe
  stripe_checkout_id TEXT,
  stripe_payment_intent_id TEXT,

  -- Status
  status       TEXT NOT NULL DEFAULT 'active', -- active | revoked | expired

  created_at   TEXT NOT NULL DEFAULT (datetime('now')),
  updated_at   TEXT NOT NULL DEFAULT (datetime('now')),

```

```
    UNIQUE(user_id, capsule_id)
);
```

```
-- Indexes
```

```
CREATE INDEX idx_capsules_slug ON capsules(slug);
CREATE INDEX idx_capsules_category ON capsules(category);
CREATE INDEX idx_capsules_downloads ON capsules(downloads DESC);
CREATE INDEX idx_releases_capsule_version ON releases(capsule_id, version);
CREATE INDEX idx_licenses_user_capsule ON licenses(user_id, capsule_id);
```

6.3 Drizzle スキーマ (TypeScript)

```
// src/db/schema.ts
import { sqliteTable, text, integer } from "drizzle-orm/sqlite-core";

export const publishers = sqliteTable("publishers", {
  id: text("id").primaryKey(),
  handle: text("handle").unique().notNull(),
  userId: text("user_id").notNull(),
  githubId: integer("github_id").unique().notNull(),
  authorDid: text("author_id").unique().notNull(),
  verified: integer("verified", { mode: "boolean" }).default(false).notNull(),
  createdAt: text("created_at")
    .notNull()
    .default(sql`(datetime('now'))`),
  updatedAt: text("updated_at")
    .notNull()
    .default(sql`(datetime('now'))`),
});

export const capsules = sqliteTable("capsules", {
  id: text("id").primaryKey(),
  publisherId: text("publisher_id")
    .notNull()
    .references(() => publishers.id),
  slug: text("slug").unique().notNull(),
  name: text("name").notNull(),
  description: text("description").notNull().default(""),
  category: text("category").notNull().default("other"),
  type: text("type").notNull().default("app"),
  homepage: text("homepage"),
  repository: text("repository"),
  licenseSpdx: text("license_spdx"),
  price: integer("price").notNull().default(0),
  currency: text("currency").notNull().default("usd"),
  stripeProductId: text("stripe_product_id"),
  downloads: integer("downloads").notNull().default(0),
  createdAt: text("created_at")
    .notNull()
    .default(sql`(datetime('now'))`),
  updatedAt: text("updated_at")
    .notNull()
    .default(sql`(datetime('now'))`),
});
```

```
export const releases = sqliteTable("releases", {
  id: text("id").primaryKey(),
  capsuleId: text("capsule_id")
    .notNull()
    .references(() => capsules.id),
  version: text("version").notNull(),
  artifactKey: text("artifact_key").notNull(),
  contentHash: text("content_hash").notNull(),
  sizeBytes: integer("size_bytes").notNull(),
  manifest: text("manifest").notNull(),
  capabilities: text("capabilities").notNull().default("[]"),
  signature: text("signature").notNull(),
  signatureStatus: text("signature_status").notNull().default("verified"),
  status: text("status").notNull().default("published"),
  createdAt: text("created_at")
    .notNull()
    .default(sql` (datetime('now')) `),
});
```

```
export const licenses = sqliteTable("licenses", {
  id: text("id").primaryKey(),
  userId: text("user_id").notNull(),
  capsuleId: text("capsule_id")
    .notNull()
    .references(() => capsules.id),
  stripeCheckoutId: text("stripe_checkout_id"),
  stripePaymentIntentId: text("stripe_payment_intent_id"),
  status: text("status").notNull().default("active"),
  createdAt: text("created_at")
    .notNull()
    .default(sql` (datetime('now')) `),
  updatedAt: text("updated_at")
    .notNull()
    .default(sql` (datetime('now')) `),
});
```

7. API エンドポイント

7.1 新規 API (/v1/)

7.1.1 Capsules (Read)

Method	Path	Auth	Description
GET	/v1/capsules	Optional	一覧 + 検索
GET	/v1/capsules/:id	Optional	詳細取得
GET	/v1/capsules/:id/download	Conditional	ダウンロード URL 取得

GET /v1/capsules — 一覧 + 検索

Query Parameters:

Param	Type	Default	Description
q	string	—	検索クエリ (スペース区切り AND)
category	string	—	カテゴリフィルタ
limit	int	20	取得件数 (max: 50)
cursor	string	—	ページネーションカーソル

Response 200 :

```
{
  "capsules": [
    {
      "id": "01HQXYZ...",
      "slug": "my-capsule",
      "name": "My Capsule",
      "description": "A great capsule",
      "category": "tools",
      "type": "app",
      "price": 0,
      "currency": "usd",
      "publisher": {
        "handle": "ato-core",
        "author_id": "did:key:z6Mk...",
        "verified": true
      },
      "latest_version": "1.2.0",
      "downloads": 1234,
      "created_at": "2026-02-01T00:00:00Z",
      "updated_at": "2026-02-07T00:00:00Z"
    }
  ],
  "next_cursor": "eyJpZCI6..."
}
```

GET /v1/capsules/:id — 詳細取得

Response 200 :


```
{
  "id": "01HQXYZ...",
  "slug": "my-capsule",
  "name": "My Capsule",
  "description": "A great capsule",
  "category": "tools",
  "type": "app",
  "homepage": "https://example.com",
  "repository": "https://github.com/user/repo",
  "license_spdx": "MIT",
  "price": 0,
  "currency": "usd",
  "publisher": {
    "handle": "ato-core",
    "author_did": "did:key:z6Mk...",
    "verified": true
  },
  "releases": [
    {
      "version": "1.2.0",
      "content_hash": "blake3:a1b2c3...",
      "size_bytes": 1024000,
      "capabilities": ["local-first", "direct-fs"],
      "signature_status": "verified",
      "status": "published",
      "created_at": "2026-02-07T00:00:00Z"
    }
  ],
  "downloads": 1234,
  "created_at": "2026-02-01T00:00:00Z"
}
```

GET /v1/capsules/:id/download — ダウンロード

Query Parameters:

Param	Type	Default	Description
version	string	latest	ダウンロードするバージョン

Response:

条件	Status	Body
無料アプリ	302	Location: <R2 Presigned GET URL (TTL 300s)>
有料 + ライセンスあり	302	Location: <R2 Presigned GET URL (TTL 300s)>
有料 + ライセンスなし	402	{ "error": "payment_required", "checkout_url": "... " }
未認証 + 有料	401	{ "error": "auth_required" }
存在しない	404	{ "error": "not_found" }

7.1.2 Publishing (Write)

Method	Path	Auth	Description
POST	/v1/capsules	Publisher	メタデータ登録
POST	/v1/capsules/:id/releases	Publisher	リリース作成 + Upload URL 取得
POST	/v1/capsules/:id/releases/:version/finalize	Publisher	アップロード完了通知

POST /v1/capsules — メタデータ登録

Request:

```
{
  "slug": "my-capsule",
  "name": "My Capsule",
  "description": "A great capsule",
  "category": "tools",
  "type": "app",
  "price": 0,
  "currency": "usd"
}
```

Response 201 :

```
{
  "id": "01HQXYZ...",
  "slug": "my-capsule"
}
```

POST /v1/capsules/:id/releases — リリース作成

Request:

```
{
  "version": "1.0.0",
  "content_hash": "blake3:a1b2c3...",
  "size_bytes": 1024000,
  "manifest": { "...capsule.toml as JSON..." },
  "capabilities": ["local-first"],
  "signature": {
    "algorithm": "Ed25519",
    "public_key": "did:key:z6Mk...",
    "content_hash": "blake3:a1b2c3...",
    "signature": "<base64>",
    "signed_at": 1738886400
  }
}
```

Response 201 :

```
{
  "release_id": "01HQABC...",
  "upload_url": "https://<account>.r2.cloudflarestorage.com/capsules/01HQXYZ/1.0.0.sync?X-
  "upload_expires_in": 300
}
```

POST /v1/capsules/:id/releases/:version/finalize — 完了通知

Server-side validation:

1. R2 にオブジェクトが存在することを確認
2. content_hash が一致することを確認 (R2 メタデータまたは再計算)
3. releases.status を published に更新

Error:

条件	Status	Error
既存バージョンへの重複作成	409	version_exists
署名検証失敗	403	invalid_signature
OAuth + DID 不一致	403	identity_mismatch

7.1.3 Publisher

Method	Path	Auth	Description
POST	/v1/publishers/register	GitHub OAuth	Publisher 登録 (DID Binding)
GET	/v1/publishers/:handle	Optional	Publisher 情報取得

POST /v1/publishers/register

Request:

```
{
  "handle": "ato-core",
  "author_did": "did:key:z6MkhaXgBZDvotDkL5257faiztiGiC2QtKLGpbnnEGta2doK",
  "did_proof": {
    "did": "did:key:z6Mk...",
    "timestamp": "2026-02-07T00:00:00Z",
    "signature": "<base64-ed25519-signature>"
  }
}
```

Validation:

1. Authorization: Bearer <github_oauth_token> を検証
2. did_proof.did から Ed25519 公開鍵を抽出
3. did_proof.timestamp に対する did_proof.signature を検証
4. timestamp が現在時刻 ±5 分以内であることを確認
5. github_id と author_did を紐付けて publishers に INSERT

7.1.4 Commerce

Method	Path	Auth	Description
POST	/v1/store/checkout	User	Stripe Checkout Session 作成
POST	/v1/webhooks/stripe	Stripe Sig	Stripe Webhook 受信

POST /v1/store/checkout

Request:

```
{
  "capsule_id": "01HQXYZ..."
}
```

Response 200 :

```
{
  "checkout_url": "https://checkout.stripe.com/c/pay/cs_..."
}
```

POST /v1/webhooks/stripe

処理する Stripe イベント:

Event	Action
checkout.session.completed	licenses レコード作成 (status: "active")
charge.refunded	licenses.status を revoked に更新

7.2 Registry Discovery

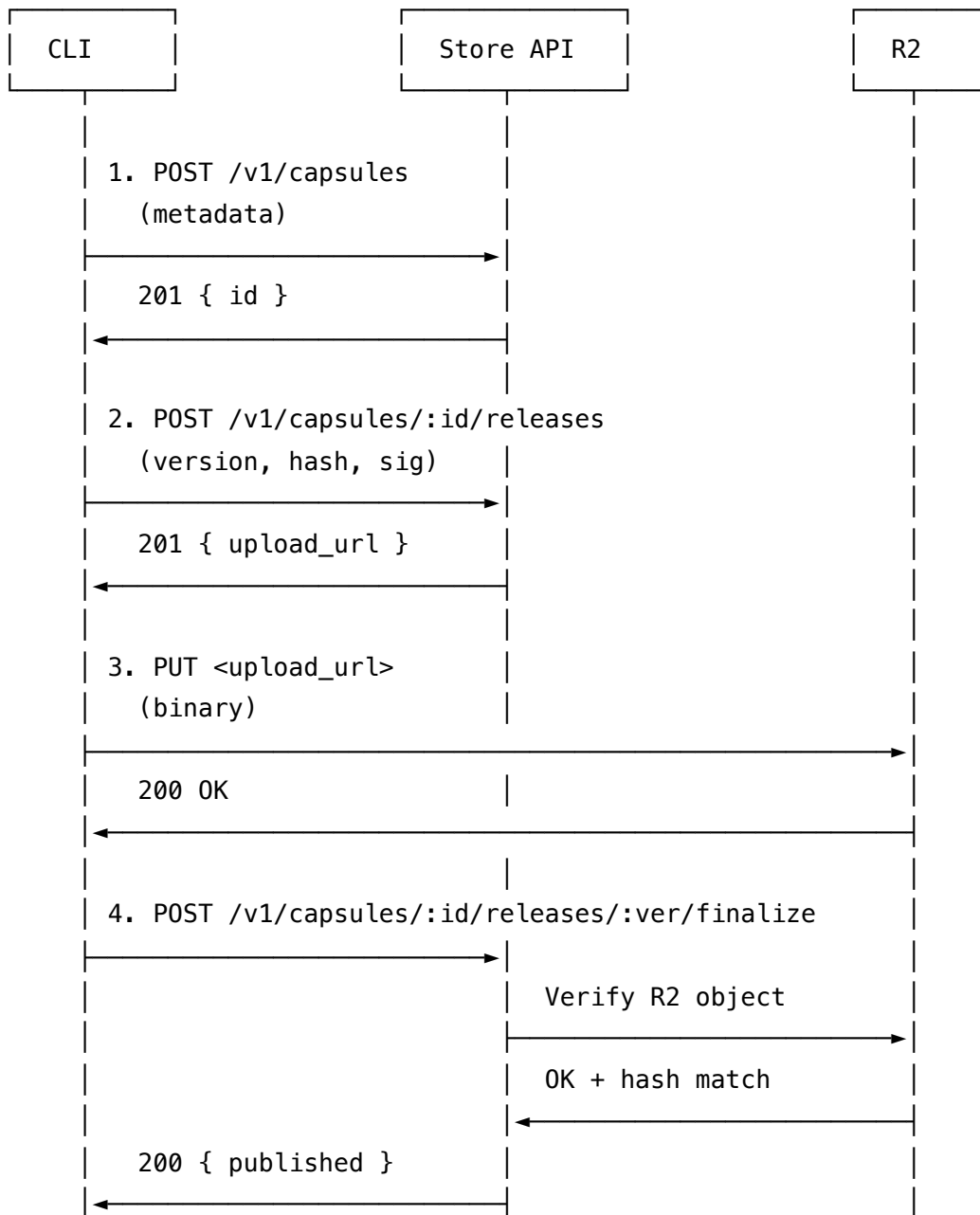
GET /.well-known/capsule.json

capsule-cli の RegistryResolver (DNS TXT → Well-known JSON → Fallback) の Well-known 発見をサポートする。

```
{
  "url": "https://store.capsule.app",
  "name": "Ato Store",
  "public_key": "did:key:z6Mk...",
  "version": "1"
}
```

8. ワークフロー仕様

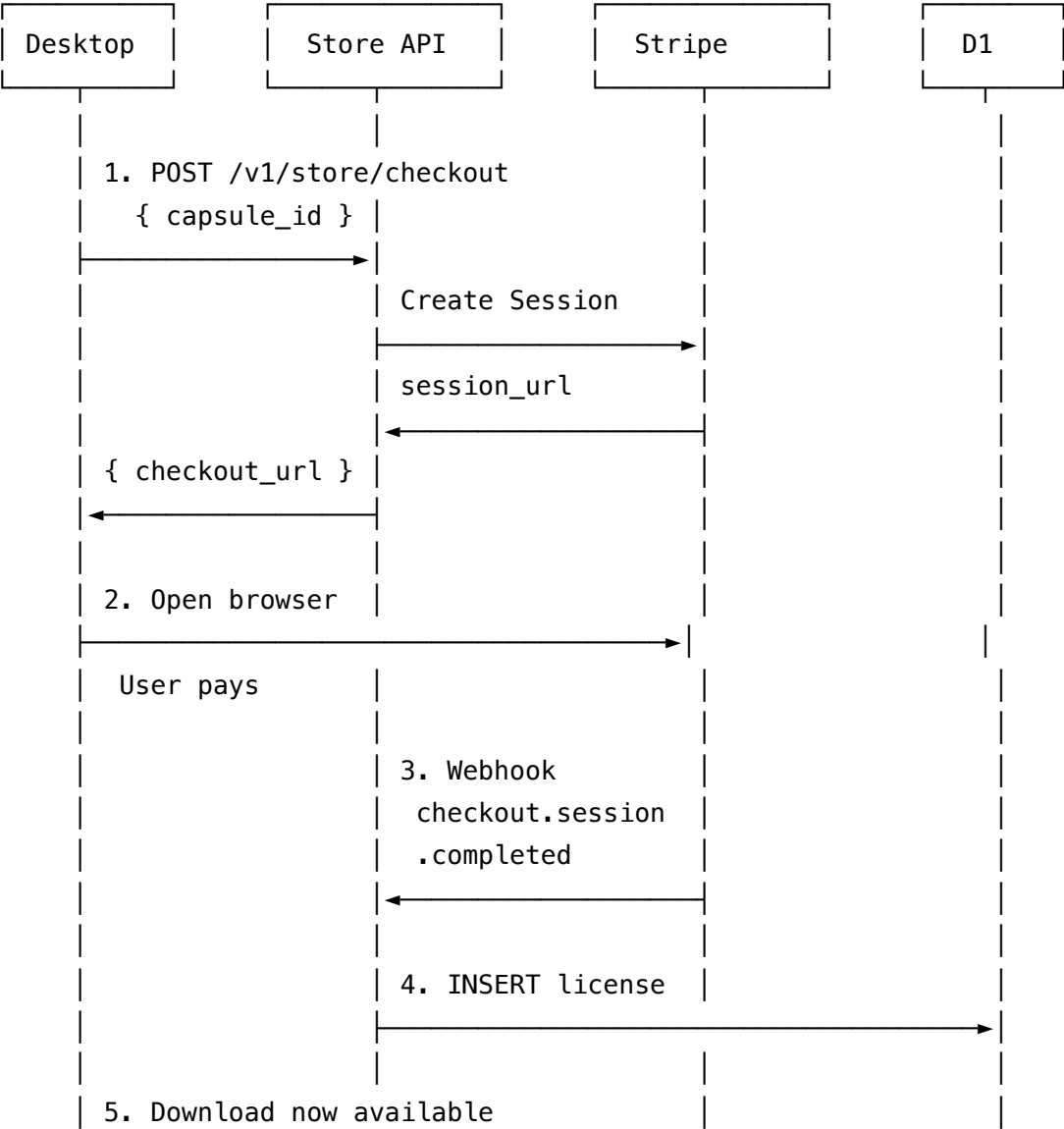
8.1 パブリッシュ (Immutable Release)



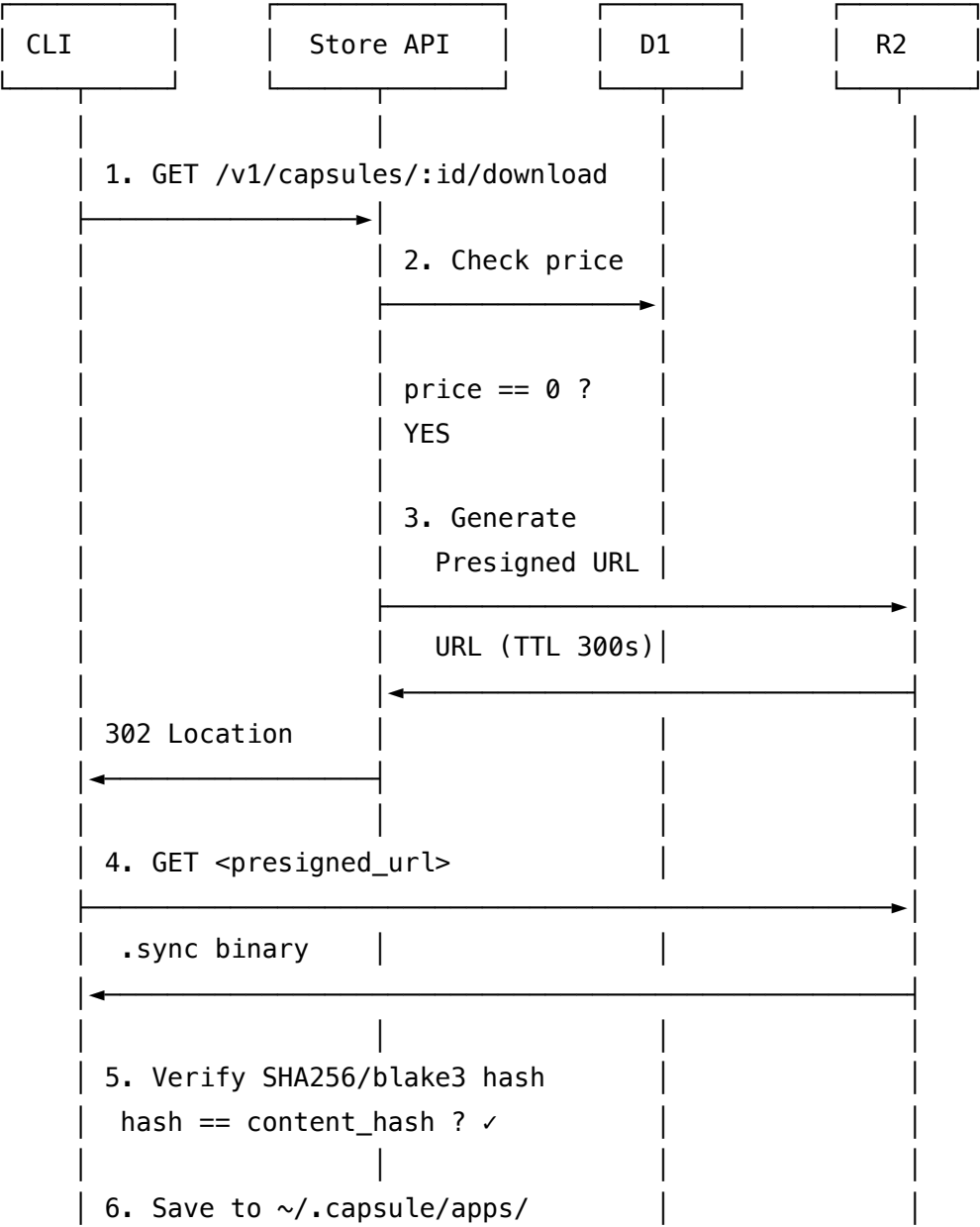
制約:

- 既存バージョン (**v1.0.0**) の上書きは不可。修正は **v1.0.1** で行う → **409 Conflict**
- `capsule.toml` の `name` パターン (`^[a-z0-9]([a-z0-9-]*[a-z0-9])?$` , max 63) に準拠

8.2 購入フロー (Stripe Checkout)



8.3 インストールと権限チェック



クライアント側の検証:

ダウンロード後、クライアントは `content_hash (blake3)` を再計算し、メタデータと照合する。不一致の場合はファイルを破棄する（サプライチェーン攻撃対策）。

9. クライアント統合

9.1 Rust Core (capsule-store-client)

capsule-cli/core/src/store_client.rs に新規モジュールとして実装する。

責務:

- Store API との HTTP 通信 (request)
- 認証トークン管理 (~/.capsule/credentials.json)
- blake3 ハッシュ検証
- Presigned URL 経由のアップロード/ダウンロード

型定義 (Rust 側):

```
// capsule-core/src/store_client.rs

#[derive(Debug, Serialize, Deserialize)]
pub struct CapsuleSummary {
    pub id: String,
    pub slug: String,
    pub name: String,
    pub description: String,
    pub category: String,
    pub r#type: String,
    pub price: u64,
    pub currency: String,
    pub publisher: PublisherInfo,
    pub latest_version: String,
    pub downloads: u64,
    pub created_at: String,
    pub updated_at: String,
}

#[derive(Debug, Serialize, Deserialize)]
pub struct PublisherInfo {
    pub handle: String,
    pub author_id: String,
    pub verified: bool,
}

#[derive(Debug, Serialize, Deserialize)]
pub struct InstallInfo {
    pub download_url: String,
    pub content_hash: String,
    pub size_bytes: u64,
}
```

CLI コマンド拡張:

Command	Description
capsule login	Device Flow で OAuth 認証 → トークン保存
capsule publish	capsule.toml を解析 → メタデータ + アーティファクトを Store に公開
capsule install <slug>	Store から Capsule をダウンロード → ~/.capsule/apps/ に保存

Command	Description
capsule search <query>	Store を検索

9.2 Tauri Frontend (React)

既存の `app_store.rs` のハードコードカタログを Store API クライアントに置換する。

Tauri Commands:

Command	Current	After
<code>list_catalog_apps</code>	ハードコード 3 件	Store API GET <code>/v1/capsules</code>
<code>get_catalog_app</code>	ハードコード	Store API GET <code>/v1/capsules/:id</code>
<code>install_capsule</code>	—	Store API download + 検証
<code>purchase_capsule</code>	—	Stripe Checkout URL 取得 + ブラウザ open

UI フロー:

1. **Store Home:** `list_catalog_apps` → `Vec<CapsuleSummary>` を描画
2. **Install:** `install_capsule(id)` → プログレスバー表示
3. **Update Badge:** Store の `latest_version` と `InstalledApp.version` を比較 → 「アップデートあり」バッジ表示
4. **Purchase:** `purchase_capsule(id)` → ブラウザで Stripe Checkout → Webhook で `licenses` 作成 → ダウンロード可能に

10. ato-coordinator からの移行

10.1 方針: 完全置換 (Option A)

Store API を正本とし、`ato-coordinator` の Registry 機能は廃止する。

10.2 移行タイムライン

Phase	期間	内容
並行運用	2 週間	Store API と coordinator 両方が動作。CLI のデフォルトは coordinator のまま
切り替え	Day 15	CLI のデフォルトエンドポイントを Store API に変更するアップデートを配信
廃止	Day 30	coordinator の Registry エンドポイントを停止

10.3 データ移行

既存の registry_apps テーブルから Store API の capsules + releases テーブルへ一括移行スクリプトを作成する。

マッピング:

coordinator (RegistryApp)	Store (capsules + releases)
id (blake3 hash)	releases.content_hash として保持
name	capsules.name
description	capsules.description
category	capsules.category
author_did	publishers.author_did (Publisher レコード要作成)
content_hash	releases.content_hash
capabilities	releases.capabilities
signature	releases.signature
signature_status	releases.signature_status
size_bytes	releases.size_bytes
downloads	capsules.downloads
file_path	R2 にアップロード → releases.artifact_key

11. プロジェクト構造

```
apps/ato-store/  
├─ src/  
│   ├── index.ts           # Hono entry point  
│   ├── bindings.ts        # Cloudflare Bindings 型定義  
│   ├── auth.ts            # Better Auth 構成  
│   ├── db/  
│   │   └─ schema.ts       # Drizzle スキーマ  
│   ├── routes/  
│   │   ├── capsules.ts    # /v1/capsules/*  
│   │   ├── publishers.ts  # /v1/publishers/*  
│   │   ├── commerce.ts    # /v1/store/checkout, webhooks  
│   │   └─ well-known.ts   # /.well-known/capsule.json  
│   ├── services/  
│   │   ├── search/  
│   │   │   ├── types.ts   # SearchService interface  
│   │   │   └─ simple-sql.ts # D1 LIKE 実装  
│   │   ├── storage.ts     # R2 Presigned URL 管理  
│   │   └─ stripe.ts       # Stripe 連携  
│   └─ middleware/  
│       └─ auth.ts         # requireAuth(), requirePublisher()  
├─ drizzle/                # Migration files  
├─ scripts/  
│   ├── seed.ts            # テストデータ投入  
│   └─ migrate-coordinator.ts # coordinator データ移行  
├─ drizzle.config.ts  
├─ package.json  
├─ tsconfig.json  
├─ wrangler.toml  
└─ vitest.config.ts
```

12. 実装ロードマップ

Phase 0: Foundation & Validation (Week 1)

- ☐ Workers + Hono + Drizzle (D1) + R2 の環境構築
- ☐ Better Auth on Workers の動作検証 (ダメなら hono/oauth-providers ヘピボット)
- ☐ Drizzle スキーマ定義 + マイグレーション

- ☐ Vitest + @cloudflare/vitest-pool-workers セットアップ
- ☐ wrangler dev ローカル環境整備

Phase 1: Read-Only API (Week 2) ← 最優先

目的: CLI チームがダウンロード機能の実装に入れるようにする

- ☐ SimpleSqlSearchService (LIKE ベース) 実装
- ☐ GET /v1/capsules (一覧 + 検索)
- ☐ GET /v1/capsules/:id (詳細)
- ☐ GET /v1/capsules/:id/download (無料アプリ向け Presigned URL)
- ☐ GET /.well-known/capsule.json
- ☐ テスト用シードデータ投入

Phase 2: Auth & Identity (Week 3)

- ☐ Better Auth: GitHub (Publisher) / Google (Buyer) OAuth
- ☐ POST /v1/publishers/register (DID Binding)
- ☐ CLI Device Flow (capsule login)
- ☐ requireAuth() / requirePublisher() ミドルウェア

Phase 3: Write & Publish (Week 4)

- ☐ POST /v1/capsules (メタデータ登録)
- ☐ POST /v1/capsules/:id/releases (Presigned PUT URL 発行)
- ☐ POST /v1/capsules/:id/releases/:version/finalize (完了通知)
- ☐ Ed25519 署名検証 (DID 公開鍵)
- ☐ ato-coordinator からのデータ移行スクリプト

Phase 4: Commerce & E2E Testing (Week 5)

- ☐ POST /v1/store/checkout (Stripe Checkout Session)
- ☐ POST /v1/webhooks/stripe (Webhook 処理)
- ☐ 有料アプリのダウンロードガード (licenses チェック)
- ☐ E2E テスト: CLI → Publish → Install → ハッシュ検証
- ☐ 負荷テスト (k6)

Phase 5: Client Integration (Week 6)

- ☐ Rust capsule-store-client モジュール
- ☐ capsule login / capsule publish / capsule search CLI コマンド

- ☐ Desktop app_store.rs のカタログ切り替え
- ☐ Desktop Install UI + プログレスバー
- ☐ Update Badge (ローカル版 vs Store 最新版の比較)

13. テスト戦略

13.1 ユニットテスト

対象	ツール	内容
Search Service	Vitest	LIKE クエリ構築、トランケーション
Auth Middleware	Vitest + Workers pool	トークン検証、ロール判定

13.2 統合テスト

対象	ツール	内容
API Routes	@cloudflare/vitest-pool-workers	D1/R2 ローカルエミュレーション上でのリクエスト/ レスポンス
Stripe Webhook	Vitest + Stripe CLI	stripe listen --forward-to でローカルテスト

13.3 E2E テスト

シナリオ	内容
Publish → Install	CLI で capsule publish → capsule install → blake3 検証
Purchase → Download	Desktop で購入 → Webhook → ダウンロード可能確認

13.4 負荷テスト

- ツール: k6 (Grafana)
- 目標: Workers の同時接続上限 (1000 req/CPU 時間) 内で 100 req/sec を安定処理
- シナリオ: 検索 70%、ダウンロード 20%、公開 10%

14. セキュリティ考慮事項

14.1 認証

- パスワード認証は**完全に無効化**
- OAuth プロバイダのみ (GitHub / Google / Apple)
- Publisher 昇格には GitHub アカウントリンクが必須

14.2 署名検証

- すべてのリリースは Ed25519 + blake3 で署名必須
- DEV_MODE 環境変数での署名スキップは **Store API では許可しない** (coordinator とは異なる)

14.3 サプライチェーン

- R2 Presigned URL の TTL は 300 秒
- クライアントは必ず content_hash を再計算して照合
- 不一致時はファイル破棄 + エラー報告

14.4 Stripe Webhook

- Stripe-Signature ヘッダーによる Webhook 署名検証必須
- Idempotency key で重複処理を防止

15. D1 制限事項と緩和策

本仕様で使用する Cloudflare D1 の制限事項:

項目	Free	Workers Paid	影響
最大 DB サイズ	500 MB	10 GB	メタデータのみなので十分
行数上限	無制限	無制限	—
最大行サイズ	2 MB	2 MB	manifest JSON は 2MB 未満であること

項目	Free	Workers Paid	影響
LIKE パターン	50 バイト	50 バイト	検索クエリを 50 バイト以内にトランケート
FTS5	非対応	非対応	Search Abstraction Layer で緩和
1 Worker 呼び出しあたりクエリ数	50	1,000	N+1 クエリに注意

日本語検索の制約:

LIKE パターン 50 バイトは UTF-8 日本語で実質 16~25 文字。Phase 2 で以下を検討:

- Cloudflare Workers AI + Vectorize (CF ネイティブ、追加コスト最小)
- Meilisearch Cloud (\$20/月)

16. 未決事項

16.1 パッケージ Yank (取り下げ)

- `releases.status` を `yanked` に設定し、新規ダウンロードを拒否
- 既存インストール済みユーザーへの影響は？
- 完全削除 vs 非表示化の判断基準

16.2 Stripe Connect (Publisher 収益分配)

- Phase 4 以降で実装
- Connected Account の KYC フロー
- 分配比率の設定 (固定 vs 可変)

16.3 Organization / Team

- Publisher を個人からチームに拡張するか？
- GitHub Organization との連携

16.4 Private Packages

- Organization 内でのみアクセス可能なパッケージ

- アクセス制御の粒度 (Team / User)

16.5 Rate Limiting

- Workers の built-in Rate Limiting vs Cloudflare WAF
- エンドポイントごとの制限値

17. 参照

- [LICENSE_SPEC.md](#) — ライセンスプロトコル (`license.sync` , PoL 検証)
- [IDENTITY_SPEC.md](#) — Identity Layer (`did:key` , Ed25519 署名)
- [TRUST_AND_KEYS.md](#) — Trust UX / Key Rotation
- [CAPSULE_CLI_SPEC.md](#) — CLI コマンド仕様 (`capsule install` , `capsule publish`)
- [DESKTOP_SPEC.md](#) — Desktop UI/UX 仕様
- [CAPSULE_SPEC.md](#) — `capsule.toml` マニフェスト仕様
- [SYNC_SPEC.md](#) — `.sync` カプセルフォーマット